

Cognizant-upgrad Training Program

Uge_dotnet_FSD_Python

Trainee Information

Name: *GOLLA AMULYA*

Email: *gollaamulya418@gmail.com*

Batch Name: *B1_Python*

Course: *uge_dotnet FSD (Python)*

Training Duration: *16th Feb 2026 – 26th May 2026*

Session Timings: *9:30 AM – 6:30 PM*

Faculty: *Kunal*

Week 8

PROBLEM STATEMENT

A startup wants to build a Food Delivery Platform using microservices.

Users should be able to:

- Register/login
- Browse restaurants
- Place food orders
- Track order status

CustomerService

Models/Customer.cs

```
namespace CustomerService.Models;
```

```
public class Customer
```

```
{
```

```
    public int Id { get; set; }
```

```
    public string Name { get; set; }
```

```
    public string Email { get; set; }
```

```
}
```

Controllers/CustomerController.cs

```
using Microsoft.AspNetCore.Mvc;
```

```
using CustomerService.Models;
```

```
namespace CustomerService.Controllers;
```

```
[ApiController]
```

```
[Route("api/customers")]
```

```
public class CustomerController : ControllerBase
```

```
{
```

```
    private static List<Customer> _customers = new();
```

```
    [HttpPost]
```

```
    public IActionResult Create(Customer customer)
```

```
    {
```

```
        if (string.IsNullOrEmpty(customer.Name))
```

```
            return BadRequest();
```

```
        _customers.Add(customer);
```

```
        return Ok(customer);
```

```
    }
```

```
    [HttpGet("{id}")]
```

```
    public IActionResult Get(int id)
```

```
    {
```

```
        var customer = _customers.FirstOrDefault(x => x.Id == id);
```

```
        if (customer == null)
```

```
            return NotFound();
```

```
        return Ok(customer);
```

```
    }
```

```
}
```

Models/Restaurant.cs

```

namespace RestaurantService.Models;
public class Restaurant
{
    public int Id { get; set; }
    public string Name { get; set; }
}

```

```

Controllers/RestaurantController.cs
using Microsoft.AspNetCore.Mvc;
using RestaurantService.Models;
namespace RestaurantService.Controllers;
[ApiController]
[Route("api/restaurants")]
public class RestaurantController : ControllerBase
{
    private static List<Restaurant> _restaurants = new();
    [HttpPost]
    public IActionResult Add(Restaurant restaurant)
    {
        _restaurants.Add(restaurant);
        return Ok(restaurant);
    }
    [HttpGet("{id}")]

    public IActionResult Get(int id)
    {
        var r = _restaurants.FirstOrDefault(x => x.Id == id);
        if (r == null)
            return NotFound();
        return Ok(r);
    }
}

```

```

Models/MenuItem.cs
namespace MenuService.Models;
public class MenuItem
{
    public int Id { get; set; }
    public int RestaurantId { get; set; }
    public string Name { get; set; }
    public double Price { get; set; }
}

```

```

Controllers/MenuController.cs
using Microsoft.AspNetCore.Mvc;
using MenuService.Models;
namespace MenuService.Controllers;
[ApiController]
[Route("api/menu")]

```

```

public class MenuController : ControllerBase
{
    private static List<MenuItem> _menu = new();
    [HttpPost]
    public IActionResult Add(MenuItem item)
    {
        if (item.Price <= 0)
            return BadRequest("Invalid Price");
        _menu.Add(item);
        return Ok(item);
    }
    [HttpGet("{restaurantId}")]
    public IActionResult Get(int restaurantId)
    {
        var items = _menu.Where(x => x.RestaurantId == restaurantId).ToList();
        return Ok(items);
    }
}

```

OrderService

ICustomerService.cs

namespace OrderService.Services;

public interface ICustomerService

```

{
    bool IsValidCustomer(int id);
}

```

IRestaurantService.cs

namespace OrderService.Services;

public interface IRestaurantService

```

{
    bool IsValidRestaurant(int id);
}

```

IMenuService.cs

namespace OrderService.Services;

public interface IMenuService

```

{
    bool IsValidMenu(int id);
}

```

Controllers/OrderController.cs

using Microsoft.AspNetCore.Mvc;

using OrderService.Services;

namespace OrderService.Controllers;

[ApiController]

[Route("api/orders")]

```

public class OrderController : ControllerBase
{
    private readonly ICustomerService _customer;
    private readonly IRestaurantService _restaurant;
    private readonly IMenuService _menu;
    public OrderController(
        ICustomerService customer,
        IRestaurantService restaurant,
        IMenuService menu)
    {
        _customer = customer;
        _restaurant = restaurant;
        _menu = menu;
    }
    [HttpPost]
    public IActionResult Create(int customerId, int restaurantId, int menuId)
    {
        try
        {
            if (!_customer.IsValidCustomer(customerId))
                return BadRequest("Invalid Customer");
            if (!_restaurant.IsValidRestaurant(restaurantId))

            return BadRequest("Invalid Restaurant");
            if (!_menu.IsValidMenu(menuId))
                return BadRequest("Invalid Menu Item");
            return Ok("Order Created Successfully");
        }
        catch
        {
            return StatusCode(500);
        }
    }
}

```

ocelot.json

```

{
  "Routes": [
    {
      "DownstreamPathTemplate": "/api/orders",
      "UpstreamPathTemplate": "/orders",
      "DownstreamHostAndPorts": [
        { "Host": "localhost", "Port": 5003 }
      ]
    }
  ]
}

```

OrderTests.cs

```

using Xunit;
using Moq;
using OrderService.Controllers;
using OrderService.Services;
using Microsoft.AspNetCore.Mvc;
namespace OrderService.Tests;

public class OrderTests
{
    [Fact]
    public void CreateOrder_Success()
    {
        var cust = new Mock<ICustomerService>();
        var rest = new Mock<IRestaurantService>();
        var menu = new Mock<IMenuService>();
        cust.Setup(x => x.IsValidCustomer(1)).Returns(true);
        rest.Setup(x => x.IsValidRestaurant(1)).Returns(true);
        menu.Setup(x => x.IsValidMenu(1)).Returns(true);
        var controller = new OrderController(cust.Object, rest.Object, menu.Object);
        var result = controller.Create(1, 1, 1);
        Assert.IsType<OkObjectResult>(result);
    }
    [Fact]
    public void InvalidCustomer_ShouldReturnBadRequest()
    {
        var cust = new Mock<ICustomerService>();
        cust.Setup(x => x.IsValidCustomer(It.IsAny<int>())).Returns(false);
        var controller = new OrderController(
            cust.Object,
            new Mock<IRestaurantService>().Object,
            new Mock<IMenuService>().Object);
        var result = controller.Create(1, 1, 1);
        Assert.IsType<BadRequestObjectResult>(result);
    }
}

```

```

CustomerService.Tests
using Xunit;
using Microsoft.AspNetCore.Mvc;
using CustomerService.Controllers;
using CustomerService.Models;
public class CustomerTests
{
    [Fact]
    public void CreateCustomer_Success()
    {
        var controller = new CustomerController();
        var result = controller.Create(new Customer { Id = 1, Name = "John" });
    }
}

```

```

Assert.IsType<OkObjectResult>(result);
}
[Fact]
public void InvalidCustomer_ShouldFail()
{
    var controller = new CustomerController();
    var result = controller.Create(new Customer { Id = 1, Name = "" });
    Assert.IsType<BadRequestResult>(result);
}
}
RestaurantService.Tests
using Xunit;
using Microsoft.AspNetCore.Mvc;
using RestaurantService.Controllers;
using RestaurantService.Models;
public class RestaurantTests
{
    [Fact]
    public void AddRestaurant_Success()

    {
        var controller = new RestaurantController();
        var result = controller.Add(new Restaurant { Id = 1, Name = "KFC" });
        Assert.IsType<OkObjectResult>(result);
    }
    [Fact]
    public void InvalidId_ShouldReturnNotFound()
    {
        var controller = new RestaurantController();
        var result = controller.Get(99);
        Assert.IsType<NotFoundResult>(result);
    }
}

MenuService.Tests
using Xunit;
using Microsoft.AspNetCore.Mvc;
using MenuService.Controllers;
using MenuService.Models;
public class MenuTests
{
    [Fact]
    public void AddMenu_Success()
    {
        var controller = new MenuController();
        var result = controller.Add(new MenuItem { Id = 1, Price = 100 });
        Assert.IsType<OkObjectResult>(result);
    }
}

```

```
[Fact]
public void InvalidPrice_ShouldFail()
{
    var controller = new MenuController();
    var result = controller.Add(new MenuItem { Price = 0 });
    Assert.IsType<BadRequestObjectResult>(result);
}
}
```

Shell scripting linux

1 Create & List

Create a directory test1 and go inside it. List contents.

```
mkdir test1
```

```
cd test1
```

2 Multiple Files

🔗 Create 3 files: a.txt, b.txt, c.txt

```
test1 % touch a.txt b.txt c.txt
```

3 Add Content

🔗 Add text "Linux Practice" into a.txt

```
test1 % echo "Linux Practice" > a.txt
```

4 View Content

```
cat a.txt
```

5 Copy File

```
test1 % cp a.txt copy_a.txt
```

6 Rename File

```
test1 % mv b.txt new_b.txt
```

7 Delete File

```
rm c.txt
```

📁 ◊ INTERMEDIATE (Directories + Move)

8 Move File

```
test1 % mkdir data
```

```
mv a.txt data/
```

9 Nested Directory

🔗 Create directory structure:

```
data % mkdir -p project/logs
```


10 List Recursively

```
data % ls -R
a.txt project
./project:
logs
./project/logs:
```

11 Check Permissions

```
data % ls -l
total 8
-rw-r--r-- 1 m*****u staff 15 Apr 12 10:27 a.txt
drwxr-xr-x 3 m*****u staff 96 Apr 12 10:35 project
```

12 Change Permission

```
data % chmod +x a.txt
```

13 Remove Write Permission

```
data % chmod -w a.txt
```

14 Overwrite File

```
data % echo "Hello World" > a.txt
```

15 Append Data

```
data % echo "Second Line" >> file.txt
```

16 Use Pipe

```
data % cat file.txt | grep "Hello"
```

17 Combine Commands

```
data % wc -l file.txt
```

18 Current User

```
data % whoami
muniganeshnaidu
```

19 System Date

```
data % date
```

20 Running Processes

```
data % ps -ef
```

21 Simple Script

```
data % nano hello.sh
#!/bin/bash
```

```
echo "Hello Linux"
```

```
data % chmod +x hello.sh  
data % ./hello.sh
```

22 Username Script

```
#!/bin/bash  
whoami
```

23 Date Script

```
#!/bin/bash  
date
```

24 Variables

```
#!/bin/bash  
name="Pardhu"  
echo "My name is $name"
```

25 Conditional

```
#!/bin/bash  
if [ -f file.txt ]  
then  
echo "File exists"  
else  
echo "File not found"  
fi
```

26 Loop

```
#!/bin/bash  
for i in {1..5}  
do  
echo $i  
done
```

27 Combine Script

```
#!/bin/bash  
user=$(whoami)  
today=$(date)  
echo "Hello $user"  
echo "Today is $today"
```

28 Final Challenge

🔗 Do this in one go:

- Create 3 files

- Add different content
- Create folder
- Move 1 file into folder
- Display content of another file

```
touch file1.txt file2.txt file3.txt && \
echo "Content 1" > file1.txt && \
echo "Content 2" > file2.txt && \
echo "Content 3" > file3.txt && \
mkdir myfolder && \
mv file1.txt myfolder/ && \
cat file2.txt
```

Python(Day 1)

1.

Take a number and print its square and cube

```
number =int(input("Enter a number"))
print("Square:",number**2)
print("Cube: ",number**3)
```

2.

Take a string and print:

↶ first character

↶ last character

```
name ="Pardhu"
print("First Character: ",name[:1])
print("Last Character: ",name[-1:])
```

3.

Print all numbers between 1–20 that are divisible by 3

```
for i in range(1, 20):
```

```
if i%3==0:
```

```
print(i)
```

4.

Take 3 numbers and print the largest

```
numbers =[]
for i in range(len(number)):
numbers.append(int(input("Enter number:")))
largest = numbers[0]
for num in numbers:
if num > largest:
largest=num
print("Largest number: ",largest)
```

5.

Create a function that returns True if number > 100, else False

```
def check_num(num):  
    if num > 100:  
        return True  
    else:  
        return False  
num =int(input("Enter a number: "))  
print(check_num(num))
```

6.

Take input and print:

☞ "Welcome Admin" if name is "admin"

☞ else "Welcome User"

name = input("Enter your name: ")

if name == "Admin":

print("Welcome Admin")

else:

print("Welcome User")

7.

Create a list and print its length without using len()

```
numbers =[10,20,30]
```

```
count =0
```

```
for num in numbers:
```

```
    count+=1
```

```
print(count)
```

8.

Print multiplication table of a number

```
number =int(input("Enter a number: "))
```

```
for i in range(1,11):
```

```
    print(number, "x", i, "=", number*i)
```

9.

Count how many numbers in list are greater than 10

```
numbers =[1,2,11,9,20,4,55,88,0,33]
```

```
count =0
```

```
for num in numbers:
```

```
    if num>10:
```

```
        count+=1
```

```
print(count)
```

10.

Take a word and print it 5 times using loop

```
name="Pardhu"  
for i in range(5):  
    print(name)
```

11.

Take a list and create a new list with only odd numbers

```
numbers=[1, 2, 3, 4, 5, 6, 7, 8, 9]  
odd_numbers=[]  
for num in numbers:  
    if num%2==1:  
        odd_numbers.append(num)  
print(odd_numbers)
```

12.

Check if a number is prime

```
number =int(input("Enter a number: "))  
if number > 1:  
    for i in range(2,number):  
        if number%i==0:  
            print("Not prime")  
            break  
    else:  
        print("Prime")  
else :  
    print("Not prime")
```

13.

Find second largest number in a list

```
numbers = [10, 20, 5, 40, 30]  
numbers.sort()  
print("Second largest:", numbers[-2])
```

14.

Take a sentence and count:

```
🔗 vowels  
🔗 consonants  
sentence = input("Enter a sentence: ")  
vowels = 0  
consonants = 0  
for char in sentence:  
    if char.lower() in "aeiou":  
        vowels += 1  
    elif char.isalpha():
```

```
consonants += 1
print("Vowels:", vowels)
print("Consonants:", consonants)
```

15.

Create dictionary:

☞ keys = numbers (1–5)

☞ values = cube of numbers

```
cube_dict = {i: i**3 for i in range(1,6)}
print(cube_dict)
```

16. Replace all spaces in string with _

```
string = input("Enter a string with sapces: ")
new_string=string.replace(" ", "_")
print(new_string)
```

17.

Generate 5 random numbers and store in list

```
import random
numbers=[]
for i in range(1,6):
    numbers.append(random.randint(1,100))
print(numbers)
```

18.

Write a program:

☞ count how many times a number appears in list

```
numbers =[]
for i in range(1,6):
    num =int(input("Enter number: "))

numbers.append(num)
target = int(input("Enter target number: "))
count =0
for num in numbers:
    if num == target:
        count+=1
print(f"Count: {count}")
```

19.

Read a file and print only lines that contain word "Python"

```
with open("lines.txt" ,"w")as file:
    file.write("I love Python\n")
    file.write("Java is also good\n")
    file.write("Python is easy\n")
```

```
with open("lines.txt" , "r")as files:
for line in files:
if "Python" in line:
```

20.

Handle exception:

☞ user enters string instead of number → print custom error

try:

```
num = int(input("Enter a number: "))
```

```
print("You Enter: ",num)
```

except:

```
print("Enter a number ,not string")
```

21.

Check if two strings are anagrams

☞ ("listen", "silent")

```
word1 = input("Enter a word: ").lower().replace(" ","")
```

```
word2 = input("Enter a word: ").lower().replace(" ","")
```

```
if sorted(word1) == sorted(word2):
```

```
print("Anagram")
```

else:

```
print("Not anagram")
```

22.Flatten a list:

[1, [2,3], [4,5]] → [1,2,3,4,5]

```
numbers = [1,2,[3,4],[2],6,7]
```

```
result =[]
```

```
for item in numbers:
```

```
if type(item)==list:
```

```
for num in item:
```

```
result.append(num)
```

else:

```
result.append(item)
```

```
print(result)
```

23.Create dictionary from two lists:

```
keys = ["a","b","c"]
```

```
values = [1,2,3]
```

```
keys=["a","b","c"]
```

```
values=[1,2,3]
```

```
result={}
```

```
for i in range(len(keys)):
```

```
result[keys[i]]=values[i]
```

```
print(result)
```

```
key=["a","b","c"]
value=[1,2,3]
dictionary = dict(zip(key,value))
print(dictionary)
```

24.Find all duplicate elements in a list

```
numbers =[1, 2, 3, 2, 4, 5, 3, 6]
duplicate=[]
for i in range(len(numbers)):
for j in range(i+1,len(numbers)):
if numbers[i]== numbers[j] and numbers[i] not in
duplicate:
duplicate.append(numbers[i])
print("Duplicate",duplicate)
```

25.Sort a list without using sort()

```
numbers=[5, 2, 9, 1, 7]
for i in range(len(numbers)):
for j in range(i+1,len(numbers)):
if numbers[i]>numbers[j]:
numbers[i], numbers[j] = numbers[j], numbers[i]
print(numbers)
```

26.Create a program:

```
☞ reverse each word in a sentence
☞ "hello world" → "olleh dlrow"
sentence = input("Enter a sentence: ")
words = sentence.split() # split into words
result = ""
for word in words:
result += word[::-1] + " "

print(result.strip())
```

27.Write a lambda function:

```
☞ check if number is even or odd

check = lambda num: "Even" if num%2==0 else "odd"
print(check(10))
print(check(7))
```

28.Read CSV and:

```
☞ print rows where age > 25
import csv
import csv
with open("data.csv", "w", newline="") as file:
writer = csv.writer(file)
```



```
# header
writer.writerow(["name", "age", "city"])
# rows
writer.writerow(["John", 22, "NY"])
writer.writerow(["Alice", 30, "LA"])
writer.writerow(["Bob", 28, "Chicago"])
with open("data.csv", "r") as file:
    reader = csv.DictReader(file)
    for row in reader:
        if int(row["age"]) > 25:
            print(row)
29.
Create nested dictionary and:
☞ print only values using loop
dict = {
    "student1": {"name": "Pardhu", "age": 22},
    "student2": {"name": "Ganehs", "age": 23},

    "student3": {"name": "Hari", "age": 21}
}
for student in dict.values():
    for value in student.values():
        print(value)
```

30. Take list and:

☞ separate into 2 lists:

- even numbers
- odd numbers

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
even=[]
odd=[]
for num in numbers:
    if num%2==0:
        even.append(num)
    else:
        odd.append(num)
print(even)
print(odd)
```

Python(Day-2)

1.

Create a class Vehicle with:

- attributes: brand, speed
- method: move() → returns "Vehicle is moving"

Create a subclass Car that:

- overrides move() → returns "Car is moving fast"

🔗 Create object and call method.

```
class Vehical:
def __init__(self,brand,speed):
self.brand =brand
self.spped = speed
def move(self):
return "Vechial is moving"
class Car(Vehical):
def move(self):
return "Car is moving"
c=Car("Swift",100)
print(c.move())
```

2.

Create a class Employee:

- attributes: name, salary
- method: display_details()

Create subclass Manager:

- add attribute: department
- override method to include department

```
class Employee:
def __init__(self,name,salary):
self.name=name
self.salary=salary
def details_displayed(self):
print("Name",self.name)
print("Salary",self.salary)
class Manager(Employee):
def __init__(self,name,salary,department):
super().__init__(name,salary)
self.department=department
def details_displayed(self):
print("Name:", self.name)
print("Salary:", self.salary)
print("Department:", self.department)
```

```
c=Manager("Pardhu",50000,"IT")
c.details_displayed()
```

3.

Create a class Shape:

- method area() → returns 0

Create:

- Rectangle (length, width)
- Circle (radius)

🔗 Override area() in both.

```

class Shape:
def area(self):
return 0
class Rectangle(Shape):
def __init__(self,length,width):
self.length=length
self.width=width
def area(self):
return self.length*self.width
class Circle(Shape):
def __init__(self,radius):
self.radius = radius
def area(self):
return self.radius*self.radius
r=Rectangle(4,5)
c=Circle(3)
print(r.area())
print(c.area())

```

4.

Create a class BankAccount:

- attributes: account_number, balance

- method: deposit(), withdraw()

☞ Add validation:

- cannot withdraw more than balance

```

class BankAccount:
def __init__(self, account_number, balance):
self.account_number = account_number
self.balance = balance
def deposit(self, amount):
if amount > 0:
self.balance += amount
print("Deposited:", amount)
else:
print("Invalid deposit amount")
def withdraw(self, amount):
if amount > self.balance:
print("Insufficient balance ")
elif amount <= 0:
print("Invalid withdraw amount")
else:
self.balance -= amount
print("Withdrawn:", amount)
def display_balance(self):
print("Current Balance:", self.balance)

```

```
acc = BankAccount("12345", 1000)
```

```
acc.deposit(500)
acc.withdraw(200)
acc.withdraw(2000)
acc.display_balance()
```

◇ Encapsulation & Special Methods

5.

Create a class Student:

- private attribute: __marks
- method: set_marks() (validation: 0–100)
- method: get_marks()

class Student:

```
def __init__(self):
```

```
    self.__marks=0
```

```
def set_marks(self,marks):
```

```
    if 0<= marks <=100:
```

```
        self.__marks = marks
```

```
    else:
```

```
        print("Invalid marks")
```

```
def get_marks(self):
```

```
    return self.__marks
```

```
s=Student()
```

```
s.set_marks(150)
```

```
print(s.get_marks())
```

6.Create a class Book:

- attributes: title, author
- override __str__() → "Title by Author"

class Book:

```
def __init__(self, title, author):
```

```
    self.title = title
```

```
    self.author = author
```

```
def __str__(self):
```

```
    return f"{self.title} by {self.author}"
```

```
b = Book("Python Basics", "John")
```

```
print(b)
```

7.Create a class CartItem:

- attributes: name, price, quantity
- override __repr__() to display full info

class CartItem:

```
def __init__(self, name, price, quantity):
```

```
    self.name = name
```

```
    self.price = price
```

```
    self.quantity = quantity
```

```

def __repr__(self):
    return f"CartItem(name='{self.name}', price={self.price},
    quantity={self.quantity})"
item1 = CartItem("Laptop", 50000, 2)
item2 = CartItem("Mouse", 500, 3)

print(item1)
print(item2)

```

◇ Iterators & Generators

8.

Create a generator that:

- yields squares of numbers from 1 to n

```

def square_generator(n):
    for i in range(1, n + 1):
        yield i * i

```

```

for num in square_generator(5):
    print(num)

```

9.

Create a generator for:

- even numbers up to n

```

def even_number(n):
    for i in range(1, n+1):
        if i%2==0:
            yield i
for num in even_number(6):
    print(num)

```

10.

Create a custom iterator class:

- that iterates over a list in reverse order

```

class Reverseliterator:
    def __init__(self, data):
        self.data = data
        self.index = len(data) - 1 # start from last
    def __iter__(self):
        return self
    def __next__(self):
        if self.index < 0:
            raise StopIteration
        value = self.data[self.index]
        self.index -= 1
        return value

```

```
numbers = [1, 2, 3, 4, 5]
rev = Reverseliterator(numbers)
```

```
for num in rev:
    print(num)
```

◇ map, filter, reduce

11.

Given a list of numbers:

☞ Use map() to:

- convert all numbers into their cubes

```
numbers=[1,2,3,4,5]
cube = list(map(lambda x:x**3 ,numbers))
print(cube)
```

12.

Given a list of names:

☞ Use filter() to:

- return only names with length > 5

```
names=["Pardhu","Nani","Ganesh","Hari"]
result=list(filter(lambda name:len(name)>5,names))
print(result)
```

13.

Given a list:

☞ Use reduce() to:

- find product of all elements

```
from functools import reduce
numbers=[1,2,3,4,5]
product = reduce(lambda x,y: x*y,numbers)
print(product)
```

Decorators

14.

Create a decorator:

- that prints execution time of a function

☞ Apply it to a function that runs a loop

```
import time
def timer_decorator(func):
    def wrapper():
        start = time.time()
        func()
        end = time.time()
```

```

print("Execution time ",end-start,"seconds")
return wrapper
@timer_decorator
def run_loop():
for i in range (1000000):
pass
run_loop()

```

15.

Create a decorator:

- that checks if user is "admin"
- only then allows function execution
- ↳ Otherwise print "Access Denied"

```

def admin_required(func):
def wrapper(user):
if user == "admin":
return func(user)
else:
print("Access Denied ")
return wrapper

```

Function

```

@admin_required
def delete_data(user):
print("Data deleted successfully ")

```

Test

```

delete_data("admin")
delete_data("guest")

```

16.

Create a JSON file with:

- multiple users (name, email)
- ↳ Read file and:
- print only emails ending with "@gmail.com"

```

import json
# Read file
with open("users.json", "r") as file:
users = json.load(file)

# Filter and print
for user in users:
if user["email"].endswith("@gmail.com"):
print(user["email"])

```

17.

Write a regex program to:

- extract all email addresses from a string

```
import re
```

```
text = "Contact us at support@gmail.com, info@yahoo.com or  
admin@company.org"
```

```
emails = re.findall(r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}", text)
```

```
print(emails)
```